TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 5

Docker setup check, install, hello-world, nginx

Phase 1 — Foundations · Weeks 1–6 · Modules 00–02

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. Module 02 — Docker Installation and First Containers: overview
2. 02.00 Setup Check
3. 02.01 Install Docker Engine
4. 02.02 Your First Container
5. 02.03 Running a Real Image

Module 02 — Docker Installation and First Containers

What you'll be able to do after this module

- Install Docker Engine from the official Docker apt repository, including the GPG key and repository setup
- Add your user to the `docker` group so Docker commands run without `sudo`
- Run `docker run hello-world` and explain the pull, create, run, and exit sequence Docker performs
- Run an nginx container in detached mode with a port mapping and reach it from a browser
- Distinguish images from containers and clean up both with `docker rm`, `docker rmi`, and the prune commands
- Manage and inspect containers and images from the VS Code Docker sidebar

Lessons and objectives

#	Lesson	Learning objective
00	Setup Check	Verify the Ubuntu environment is ready for Docker installation by checking Linux, sudo, the curl and gpg packages, network reachability of Docker Hub, and free disk space.
01	Install Docker Engine	Install Docker Engine from the official Docker apt repository and add your user to the docker group so `docker info` works without sudo.
02	Your First Container	Run the hello-world container and explain the six steps Docker performed, from pulling the image off Docker Hub to the container exiting.
03	Running a Real Image	Run an nginx web server as a detached, port-mapped, named container, reach it from a browser, and inspect it with `docker logs` and `docker exec`.
04	Images vs Containers	Distinguish images from containers by running multiple containers from one image, then clean up both with `docker rm`, `docker rmi`, and the prune commands.
05	The VS Code Docker Sidebar	Manage containers and images from the VS Code Docker sidebar, including stopping, removing, viewing logs, and attaching a shell via the right-click menu.
06	Final Checkpoint	Demonstrate module mastery by running, verifying, and cleaning up a named detached nginx container end-to-end without instructor help.

Each lesson is one learning object with a paired lab. Work in order — completing a lab unlocks the next lesson. The module ends with a **Check on Learning** quiz and a hands-on final checkpoint.

Learning objective: Verify the Ubuntu environment is ready for Docker installation by checking Linux, sudo, the curl and gpg packages, network reachability of Docker Hub, and free disk space.

Lesson 00: Setup Check

By the end, you will have confirmed your environment is ready for Docker installation.

~5 minutes

---

1. Open VS Code and connect to your Ubuntu environment (WSL or SSH from Module 00).

2. Open the integrated terminal (`` Ctrl+ ``).

3. Confirm you're in Linux.

```
uname -s
```

```
Linux
```

4. Confirm you have `sudo`.

```
sudo whoami
```

```
root
```

(After your password.)

5. Confirm Module 01's prereq packages are installed.

```
which curl gpg
```

```
/usr/bin/curl
/usr/bin/gpg
```

6. Confirm Module 01's notes file exists.

```
ls -la ~/docker-course/notes/notes.txt
```

You should see the file. If you don't, you didn't finish Module 01 — go back and finish it before continuing.

7. Confirm Docker is **not** already installed.

```
which docker
```

(No output expected — we're about to install it. If output appears, raise your hand. We need to know what's there before we install on top of it.)

8. Confirm you have internet (Docker downloads will need it).

```
curl -s -o /dev/null -w "%{http_code}\n" https://hub.docker.com
```

```
200
```

(Or `301` / `302` — anything 2xx or 3xx means you can reach Docker Hub.)

9. Confirm at least 2 GB of free disk space.

```
df -h ~ | tail -n 1
```

Look at the "Avail" column — it should be at least 2G.

Checkpoint

```
sudo whoami && which curl gpg && ls ~/docker-course/notes/notes.txt
```

You should see `root`, two `/usr/bin/...` lines, and the path to your notes file.

Lab

Objective: Confirm your environment is ready for the Docker install — Linux, sudo, Module 01's tools and notes file, network access to Docker Hub, and free disk.

Task: Work through the nine checks in this lesson, top to bottom. Fix anything that fails before moving on — don't install onto a broken base.

Submit: Paste the output of the Checkpoint command — `sudo whoami && which curl gpg && ls ~/docker-course/notes/notes.txt` — into D2L.

Pass criteria:

- [ ] Output shows `root`.
- [ ] Output shows `/usr/bin/curl` and `/usr/bin/gpg`.
- [ ] Output shows the path to `~/docker-course/notes/notes.txt` with no "No such file" error.
- [ ] `which docker` (step 7) printed nothing — Docker is not installed yet.

Unlocks: 01-install-docker.md

Next up: 01-install-docker.md — Install Docker Engine

> **Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Install Docker Engine from the official Docker apt repository and add your user to the docker group so `docker info` works without sudo.

Lesson 01: Install Docker Engine

By the end, you will have Docker Engine installed from the official Docker apt repository, and you will be in the docker group.

~25 minutes of typing

> We are installing **Docker Engine** (the Linux daemon) — not Docker Desktop. Docker Desktop is a separate Windows/Mac product. On Linux (including WSL), Docker Engine is what you want.

---

1. Start from home.

```
cd ~
```

2. Update the package list.

```
sudo apt update
```

3. Add Docker's official GPG key. This proves the packages you download are really from Docker.

```
sudo install -m 0755 -d /etc/apt/keyrings
```

4. Download the key.

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
```

5. Make the key file readable by everyone.

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

6. Confirm the key is in place.

```
ls -la /etc/apt/keyrings/docker.asc
```

You should see the file.

7. Add the Docker apt repository to your sources list. This is one long command — type it carefully.

```
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(.
/etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
```

8. Confirm the repo file was created.

```
cat /etc/apt/sources.list.d/docker.list
```

```
deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu jammy stable
```

(Architecture and codename will vary.)

9. Update apt again — this time it'll see Docker's repo.

```
sudo apt update
```

You should see a line mentioning `download.docker.com`.

10. Install Docker Engine and the standard plugins.

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

(This will take a minute or two.)

11. Confirm Docker is installed.

```
docker --version
```

```
Docker version 24.0.7, build afdd53b
```

(Version will vary.)

12. Confirm the Docker daemon is running. **WSL users:** if `systemctl` fails with "System has not been booted with systemd," skip to step 13.

```
sudo systemctl status docker
```

You should see `Active: active (running)`. Press `q` to exit.

13. WSL only: start the Docker service the WSL way.

```
sudo service docker start
```

```
 * Starting Docker: docker
```

14. Confirm the docker group now exists (Module 01 lesson 10 said it would after install).

```
grep docker /etc/group
```

```
docker:x:998:
```

(Number will vary. The group exists — that's the point.)

15. Add yourself to the docker group. **This is the command you previewed in Module 01 lesson 10. Now it works.**

```
sudo usermod -aG docker $USER
```

(No output means success.)

16. Refresh your group membership in this shell without logging out.

```
newgrp docker
```

17. Confirm you're in the docker group.

```
groups
```

```
your-user adm sudo docker
```

You should see `docker` in the list.

18. Test Docker without sudo.

```
docker info
```

You should see a long block of info about the Docker daemon. If you get "permission denied while trying to connect to the Docker daemon socket," your group membership didn't refresh — log out and back in, or close and reopen the integrated terminal, then try again.

19. See where Docker is installed.

```
which docker
```

```
/usr/bin/docker
```

20. Log it.

```
echo "Docker installed: $(docker --version)" >> ~/docker-course/notes/notes.txt
```

21. Verify the note.

```
tail -n 1 ~/docker-course/notes/notes.txt
```

```
Docker installed: Docker version 24.0.7, build afdd53b
```

Checkpoint

```
docker --version
```

You should see a Docker version.

```
docker info | head -n 5
```

You should see info without `sudo` and without permission errors.

```
groups | grep docker
```

You should see `docker` somewhere in the output.

Lab

Objective: Install Docker Engine from the official Docker apt repository, start the daemon, and join the docker group so `docker` runs without `sudo`.

Task: Work steps 1–21 in order — GPG key, apt repo, install, start the daemon (WSL users take the step 13 path), join the docker group, then log it.

Submit: Paste the output of `docker --version`, `docker info | head -n 5`, `groups`, and `tail -n 1 ~/docker-course/notes/notes.txt` into D2L.

Pass criteria:

- [ ] `docker --version` prints a Docker version.
- [ ] `docker info | head -n 5` ran without `sudo` and shows no "permission denied" error.
- [ ] `groups` output includes `docker`.
- [ ] The daemon is running: `sudo systemctl status docker` shows `Active: active (running)`, **or** (WSL) `sudo service docker start` succeeded and `docker info` works.
- [ ] The notes line reads `Docker installed: Docker version ...`.

Unlocks: 02-hello-world.md

Next up: 02-hello-world.md — Run your first container

> **Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Run the hello-world container and explain the six steps Docker performed, from pulling the image off Docker Hub to the container exiting.

Lesson 02: Your First Container

By the end, you will have run a container and you will understand what just happened.

~10 minutes of typing

---

1. Start from home.

```
cd ~
```

2. Run the smallest possible container.

```
docker run hello-world
```

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
...
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
...
```

3. Read what Docker just told you. The first line said "Unable to find image 'hello-world:latest' locally" — **that is not an error.** Docker checked your local machine, didn't have the image, so it pulled it from Docker Hub and ran it. From now on, that image is cached locally.

4. Run it again. Notice it's faster — no pull this time.

```
docker run hello-world
```

```
Hello from Docker!
...
```

5. Look at the image Docker downloaded.

```
docker images
```

```
REPOSITORY    TAG       IMAGE ID       CREATED         SIZE
hello-world   latest    9c7a54a9a43c   8 months ago    13.3kB
```

(IDs and dates will vary. Notice the size — 13 KB. Tiny.)

6. Look at containers. By default `docker ps` shows only running ones.

```
docker ps
```

```
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS   PORTS   NAMES
```

(Empty — hello-world ran and exited.)

7. Look at all containers, including stopped ones.

```
docker ps -a
```

```
CONTAINER ID   IMAGE         COMMAND    CREATED         STATUS
PORTS   NAMES
abc123def456   hello-world   "/hello"   30 seconds ago  Exited (0) 28 seconds ago
modest_einstein
xyz789...      hello-world   "/hello"   1 minute ago    Exited (0) 1 minute ago
funny_curie
```

(Two stopped containers — one for each time you ran the image. Names are auto-generated.)

8. Read the names Docker gave them. They're random adjective + scientist combos.

9. Get just the container IDs.

```
docker ps -aq
```

```
abc123def456
xyz789ghi012
```

10. Log it.

```
echo "First container: hello-world ran successfully" >> ~/docker-
course/notes/notes.txt
```

What just happened (read this)

When you ran `docker run hello-world`, Docker did six things:

1. Looked for a local image called `hello-world` — didn't find it.
2. Connected to **Docker Hub** (docker.io) — the default public image registry.
3. Downloaded the `hello-world` image.
4. Created a **container** from that image (a running instance of it).
5. Started the container, which ran a tiny program that printed the welcome text.
6. The program finished, so the container **exited**. The container is now stopped, but still on disk (which is why `docker ps -a` shows it).

Image = the read-only blueprint Docker downloaded. **Container** = a running (or stopped) instance of an image. We'll explore this distinction more in lesson 04.

Checkpoint

```
docker images | grep hello-world
```

You should see the hello-world image listed.

```
docker ps -a | grep hello-world
```

You should see at least one stopped hello-world container.

Lab

Objective: Run your first container and explain what `docker run` actually did — pull, create, run, exit.

Task: Work steps 1–10 — run `hello-world` twice, compare `docker ps` with `docker ps -a`, then log it.

Submit: Paste the output of `docker images | grep hello-world`, `docker ps -a | grep hello-world`, and `tail ~/docker-course/notes/notes.txt` into D2L.

Pass criteria:

- [ ] The hello-world image is listed under `docker images`.

- [ ] At least one hello-world container shows `Exited (0)` in `docker ps -a`.
- [ ] The notes tail includes `First container: hello-world ran successfully`.

Unlocks: 03-running-nginx.md

Next up: 03-running-nginx.md — Run a real image

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Run an nginx web server as a detached, port-mapped, named container, reach it from a browser, and inspect it with `docker logs` and `docker exec`.

Lesson 03: Running a Real Image

By the end, you will have run a real web server in a container and reached it from a browser.

~15 minutes of typing

> **nginx** (pronounced "engine-x") is a popular web server. Today we'll run it in a container without installing it on our machine — that's the whole point of Docker.

---

1. Start from home.

```
cd ~
```

2. Pull the nginx image first (without running it). This downloads the image and shows you the layers.

```
docker pull nginx
```

```
Using default tag: latest
latest: Pulling from library/nginx
...
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

3. See the image you just pulled.

```
docker images
```

You should see `nginx` listed alongside `hello-world`. Note the size — nginx is bigger (~140 MB).

4. Run nginx in **detached** mode (in the background) and map your machine's port 8080 to the container's port 80.

```
docker run -d -p 8080:80 --name web nginx
```

```
abc123def456...
```

(One long container ID printed. That's normal.)

5. Read that command. The flags mean:

- `-d` — **detached.** Run in the background. Without this, your terminal would be stuck showing nginx logs.
- `-p 8080:80` — **port map.** Forward port 8080 on **your machine** to port 80 **inside the container.** nginx listens on port 80 inside the container; you reach it on 8080 outside.
- `--name web` — give the container a friendly name (`web`) instead of the random one Docker would pick.
- `nginx` — the image to run.

6. Confirm nginx is running.

```
docker ps
```

```
CONTAINER ID   IMAGE   COMMAND                  CREATED         STATUS
PORTS                   NAMES
abc123...      nginx   "/docker-entrypoint.…"   10 seconds ago  Up 9 seconds
0.0.0.0:8080->80/tcp   web
```

You should see your `web` container with status `Up`.

7. Hit nginx from the same terminal.

```
curl -s http://localhost:8080 | head -n 10
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
```

You should see nginx's welcome HTML.

8. Now from your browser.

- **WSL students:** open your Windows browser, go to `http://localhost:8080`. You should see the nginx welcome page.
- **SSH students:** the VS Code Remote — SSH extension automatically forwards ports. Look at the bottom panel of VS Code — there should be a **"Ports"** tab. Click it. Port 8080 should be listed (or click "Forward a Port"). Click the globe icon next to it to open it in your local browser.
- **Native Linux students:** open your browser, go to `http://localhost:8080`.

9. See the nginx logs (what requests it has received).

```
docker logs web
```

You should see lines for each request, including your `curl` and your browser visit.

10. Tail the logs in real time. Open the page in your browser again — you'll see new lines appear.

```
docker logs -f web
```

Press **Ctrl+C** to stop tailing.

11. Get a shell **inside** the running container.

```
docker exec -it web bash
```

You're now inside the container. The prompt changes to `root@<container-id>:/#`.

12. Look around — this is a different filesystem from your host.

```
ls /
```

You'll see standard Linux directories, but it's nginx's container, not your machine.

13. See what nginx is serving.

```
ls /usr/share/nginx/html/
```

```
50x.html  index.html
```

14. Exit the container shell.

```
exit
```

You're back on your host.

15. Stop the nginx container.

```
docker stop web
```

```
web
```

16. Confirm it's stopped.

```
docker ps
```

The `web` container should be gone from this list (running containers only).

17. It's still there, just stopped.

```
docker ps -a
```

You should see `web` with status `Exited`.

18. Start it again — same container, same data.

```
docker start web
```

19. Confirm it's running again and the port mapping is preserved.

```
docker ps
curl -s http://localhost:8080 | head -n 4
```

20. Stop it for now.

```
docker stop web
```

21. Log it.

```
echo "Ran nginx container 'web' on port 8080" >> ~/docker-course/notes/notes.txt
```

Checkpoint

```
docker ps -a --filter name=web
```

You should see your `web` container (likely stopped).

```
docker images | grep nginx
```

You should see the nginx image.

You should also have seen the **nginx welcome page in your browser at http://localhost:8080** at some point during this lesson.

Lab

Objective: Run nginx in a container — detached, port-mapped, named — and reach it from a browser.

Task: Work steps 1–21 — pull the nginx image, run it as `web` on port 8080, hit it with `curl` and your browser, read the logs, get a shell inside with `docker exec`, then log it.

Submit: A screenshot of the nginx welcome page in a browser — **any of the three paths counts:** your Windows browser at localhost:8080 (WSL), the VS Code **Ports** tab's globe icon (SSH), or your local browser (native Linux) — plus a paste of `docker ps -a --filter name=web`, `docker images | grep nginx`, and `tail ~/docker-course/notes/notes.txt` into D2L.

Pass criteria:

- [ ] Screenshot shows "Welcome to nginx!" served from port 8080 (whichever browser path your setup uses).
- [ ] `docker ps -a --filter name=web` shows the `web` container (stopped is fine — you stopped it in step 20).
- [ ] `docker images | grep nginx` shows the nginx image.
- [ ] The notes tail includes `Ran nginx container 'web' on port 8080`.

Unlocks: 04-images-and-containers.md

Next up: 04-images-and-containers.md — Tell images and containers apart, clean up after yourself

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.